

DEEP LEARNING

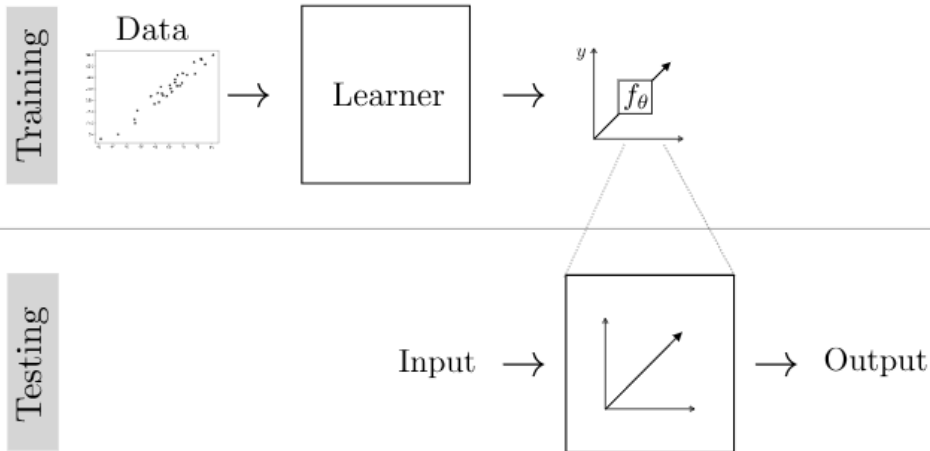
Fundamentals of Deep Learning: Part II

Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

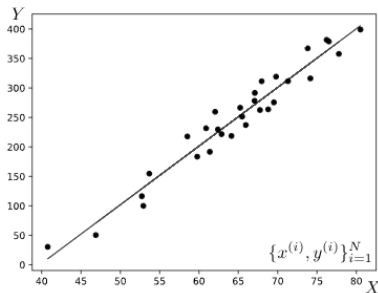
Recap

A simple learning problem called [linear least-squares regression](#) or [linear regression](#) in short.

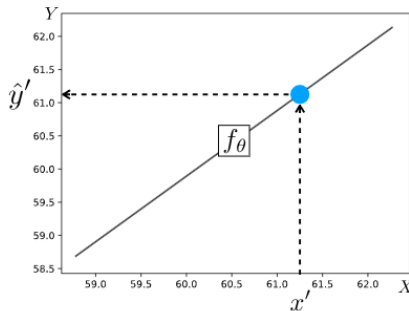


Recap

Training data

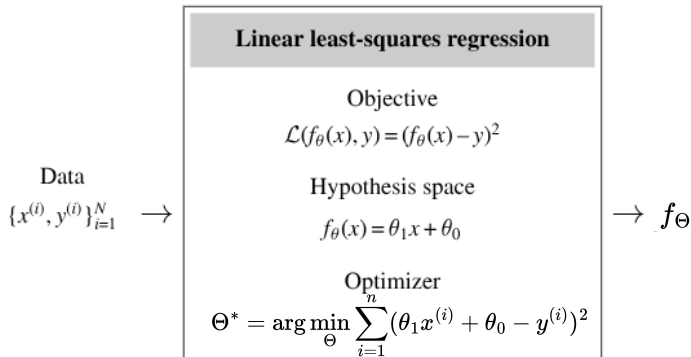


Test query



- The learned function f_θ is a straight line.
- Given a test query x' , the learned model f_θ makes a prediction $\hat{y}'$.

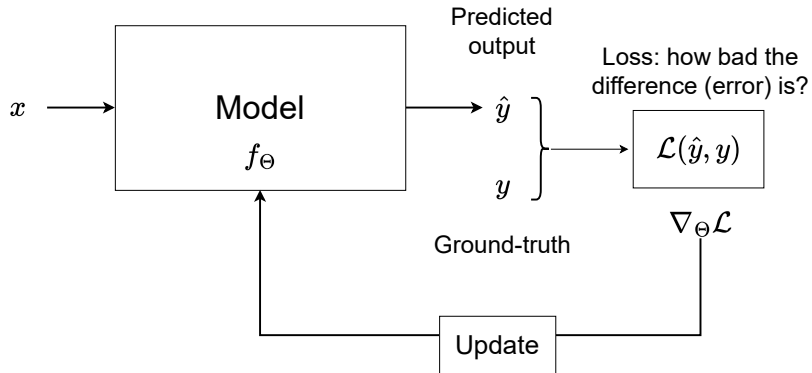
Recap



The three key ingredients in learning:

- Objective function (eg, squared error)
- Hypothesis space (eg, linear function)
- Optimizer (eg, gradient descent)

Recap



Today

- Vectorization
- More on gradient descent optimization
- Another kind of common problem called classification
- Some basic exercises in Python and PyTorch

Table of Contents

1. Vectorization

2. Gradient Descent Optimization

3. Classification Problem

Vectorization

Vectorization

- We have seen that for a single input variable x , the linear regression model is a function of parameters $\Theta = [\theta_0, \theta_1]$:

$$\hat{y} = f_{\Theta}(x) = \underbrace{\theta_0}_{\text{intercept/bias}} + \underbrace{\theta_1}_{\text{slope}} x.$$

- Extending to input containing d variables, ie, $\mathbf{x} = [x_1, x_2, \dots, x_d] \in \mathbb{R}^d$, the linear regression model is a function of parameters $\Theta = [\theta_0, \theta_1, \dots, \theta_d]$:

$$\hat{y} = f_{\Theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_d.$$

- **Question:** How can we compute $\hat{y}$ in an efficient way?
- Why do we care about efficient implementation? Because we don't have just one training data point, but we have N of them.

Vectorization

- We will avoid using for loops at all costs because its sequential. Instead we will use efficient operations like: dot products, matrix-vector product or matrix-matrix product.
- Let's try with **dot product**¹.
- Goal: we want to compute $\hat{y}$ by using the dot product between $\mathbf{x} \in \mathbb{R}^d$ and $\Theta \in \mathbb{R}^{d+1}$.
- We **can not** do the dot product $\mathbf{x}^T \Theta$ because the dimensions don't match!!
- To solve this, we add a **fictitious** (or fake) element to $\mathbf{x}$, such that $\mathbf{x} = [1, x_1, \dots, x_d] \in \mathbb{R}^{d+1}$.

¹Note that all vectors are by default column vectors

Vectorization

- Now we can do the dot product, because both vectors are of dimension $\mathbb{R}^{n+1}$.
- The dot product is can be expressed as:

$$\hat{y} = \mathbf{x}^\top \Theta = \begin{bmatrix} 1 & x_1 & \cdots & x_d \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d x_d.$$

- That's efficient, but this is just for one data point. To compute $\hat{y}$ for N data points, we still need a for loop!!

Vectorization

- To make the operations efficient for the entire training dataset, we stack each input vector $\mathbf{x} \in \mathbb{R}^{d+1}$ in a matrix as:

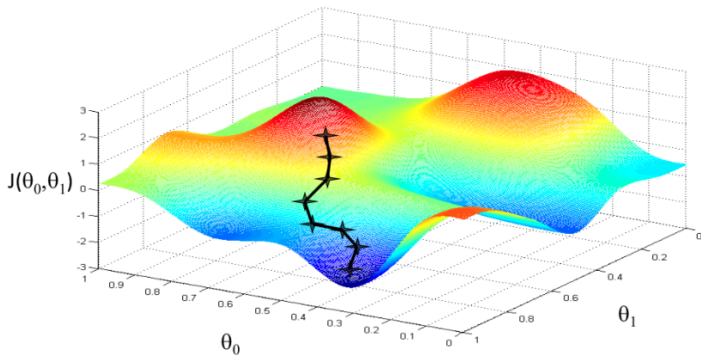
$$\mathbf{X} = \begin{bmatrix} 1 & \mathbf{x}^{(1)\top} \\ 1 & \mathbf{x}^{(2)\top} \\ \vdots & \vdots \\ 1 & \mathbf{x}^{(N)\top} \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_d^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(N)} & \cdots & x_d^{(N)} \end{bmatrix}$$

- Now, to compute $\hat{y}$ for the entire dataset all at once, we can perform matrix-vector product as:

$$\hat{\mathbf{Y}} = \mathbf{X}\Theta$$

- In DL, your job is to find operations that can be "de-loopified" and replace them with efficient matrix-vector/matrix products.

More on Gradient Descent Optimization



Gradient Descent

- So far we have seen that the weight update rule in gradient descent as:

$$\Theta \leftarrow \Theta - \alpha \nabla_{\Theta} \mathcal{L}^{(i)}$$

- This update rule is just for one training data point. We have N training data points.
- In practice, we will **average the gradients** of all the training data points (because we want to move in the average direction in the parameter space):

$$\Theta \leftarrow \Theta - \alpha \frac{1}{N} \sum_{i=1}^N \nabla_{\Theta} \mathcal{L}^{(i)}$$

- An optimization scheme that involves all the training data is called **batch gradient descent**.

Gradient Descent

- Batch gradient descent has the advantage of being numerically stable. But it has one issue!
- If $N \gg 1$ (eg, millions of training samples), then it can quickly fill the CPU/GPU RAM.
- It becomes worse with deep neural networks (we will see later why).
- For all real training, batch gradient descent is never used.

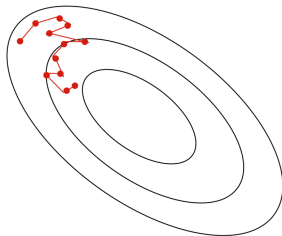
Stochastic Gradient Descent

- Instead we will use a variant of gradient descent, called **Stochastic Gradient Descent** (SGD).
- The SGD algorithm updates the weights by choosing a **single** training example. It works as follows:
 - ▶ Choose a sample i at random, ie, $(x^{(i)}, y^{(i)})$.
 - ▶ Compute the loss for that sample $\mathcal{L}(\hat{y}^{(i)}, y^{(i)})$
 - ▶ Compute the gradient for that sample: $\nabla_{\Theta} \mathcal{L}^{(i)}$
 - ▶ Apply the weight update rule: $\Theta \leftarrow \Theta - \alpha \nabla_{\Theta} \mathcal{L}^{(i)}$
- Advantage of SGD: since it uses a single sample, it is light on the memory footprint (as we doing a single dot product).

Stochastic Gradient Descent

SGD has a couple of issues:

- Gradient computed with just one sample is noisy, making SGD very noisy.



- We now lost the advantage of vectorization that we had achieved using matrix-vector product operation.

Mini-batch Gradient Descent

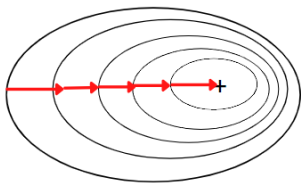
- Instead of using one training sample, we use a **medium-sized** group of M training samples (e.g., $M = 64, 128, 512$). This is called **mini-batch** gradient descent.
- The weight update rule is given as:

$$\Theta \leftarrow \Theta - \alpha \frac{1}{M} \sum_{i=1}^M \nabla_{\Theta} \mathcal{L}^{(i)}$$

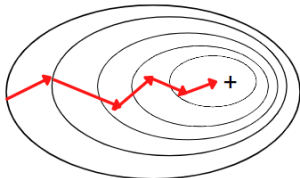
- Achieves the best of both worlds:
 - ▶ Less noisy weight updates.
 - ▶ Still allows us to use matrix-vector products $\rightarrow$ computational efficiency.
- This is the most widely used optimization strategy (even in modern deep learning systems).

Comparison of Gradient Descents

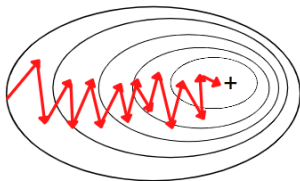
Batch Gradient Descent



Mini-Batch Gradient Descent

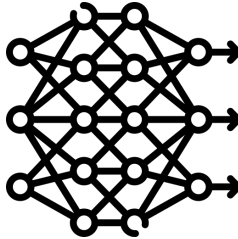
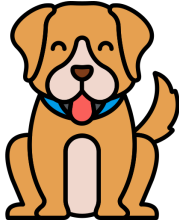


Stochastic Gradient Descent



We will come back later to gradient descent with more advanced concepts.

Classification Problem



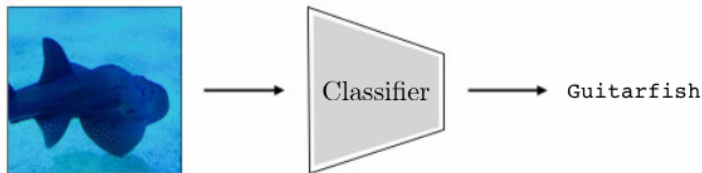
$Y = \text{"dog"}$

$Y = \text{"cat"}$

$Y = \text{"fish"}$

What is Classification Problem?

- In the regression problem, the output was a scalar value $y \in \mathbb{R}$. In our previous example, y was "number of people at the beach", which is a number.
- In the case of classification, the output is a **class label** y . The model that does classification is called a **classifier**. Below is an example of an image classifier.



- **Question:** How do we formulate the task as a learning problem?
- In other words, how do we formulate the three key ingredients: objective, hypothesis space and optimizer?

Classification Problem

- Let's say we want to build an **image classifier** that can distinguish among five animals: dolphin, grizzly bear, guitarfish, elephant and chameleon.
- First step is to collect training data and associate a **class label** (or ground-truth) to each image.



- **Question:** How can we represent the class labels and incorporate them into the learning process? Because an algorithm can only understand numbers!

Classification Problem

- We can assign unique numbers to the class labels. For example:

$$\left\{ \begin{array}{l} \text{Dolphin: } 0 \\ \text{Grizzly bear: } 1 \\ \vdots \\ \text{Chameleon: } 4 \end{array} \right.$$

- Now it appears that we can use the machinery learned in linear regression to learn a classifier, since the output y is now a number.
- Not quite, a couple of more adjustments are needed.

Classification Problem: One-hot Encoding

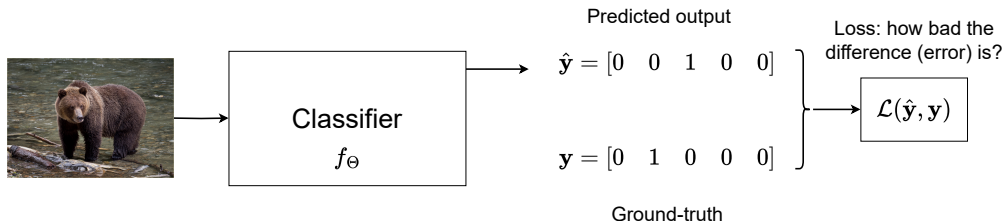
- Instead of just numbers, we will represent the class labels a K -dimensional vectors y , where $K = 5$ in our example.

$$\left\{ \begin{array}{l} \text{Dolphin: } 0 \rightarrow [1 \ 0 \ 0 \ 0 \ 0] \\ \text{Grizzly bear: } 1 \rightarrow [0 \ 1 \ 0 \ 0 \ 0] \\ \vdots \\ \text{Chameleon: } 4 \rightarrow [0 \ 0 \ 0 \ 0 \ 1] \end{array} \right.$$

- We assign a 1 at the k -th location if the class label is k , and 0 everywhere else.
- We call this representation **one-hot code**, since the vector is "hot" (or has value 1) at a single location and zero everywhere else.

Classification Problem: Objective/Loss Function

- Our goal is to learn a function f_{Θ} that outputs vectors $\hat{\mathbf{y}}$ that match the ground-truth one-hot code $\mathbf{y}$.



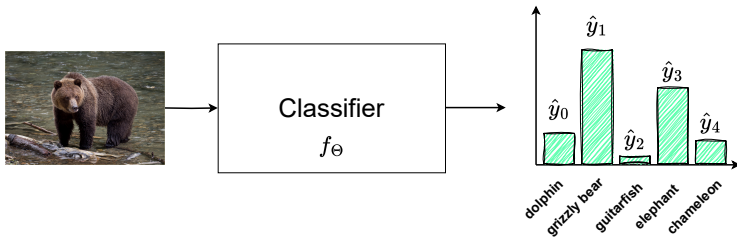
- Question:** which objective/loss function do we choose, since now we are comparing vectors?

Classification Problem: Objective/Loss Function

- For the objective function, we use [cross-entropy loss](#).
- Cross-entropy loss has the following formulation:

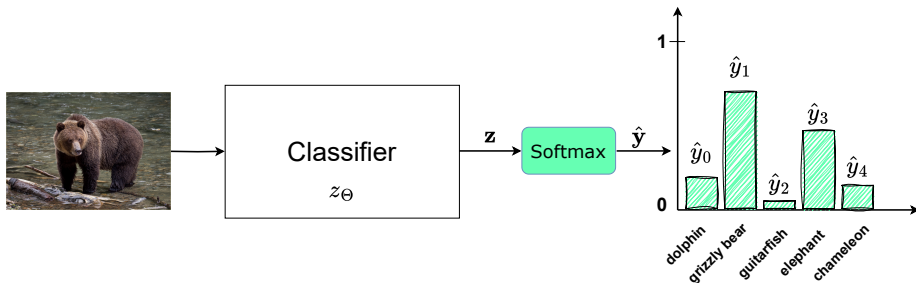
$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = H(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{k=1}^K y_k \log \hat{y}_k.$$

- Intuitively, $\hat{y}_k$ represents the probability the classifier thinks the image is an image of class k .

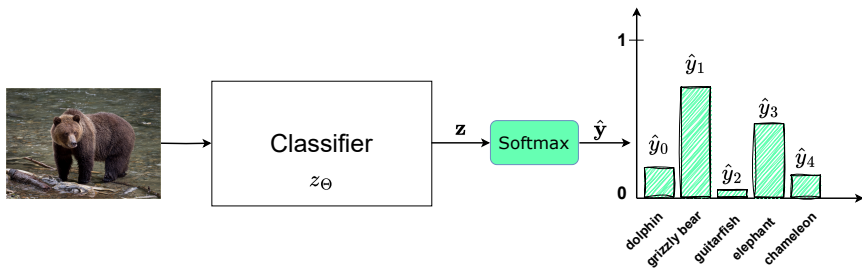


Classification Problem: Softmax Function

- Before we can call $\hat{y}_k$, ie, the elements of $\hat{\mathbf{y}}$ to be a **probability**, we need to make sure that it satisfies the two properties of **probability mass function** (pmf):
 - ▶ The value of each element in the vector should be between 0 and 1, ie, $0 \leq \hat{y}_k \leq 1$.
 - ▶ The sum of elements in the vector $\hat{\mathbf{y}}$ should be equal to 1, ie, $\sum_{k=1}^K \hat{y}_k = 1$.
- We achieve this using the **softmax** function.



Classification Problem: Softmax Function



- We get the final prediction of the classifier using the softmax function as:

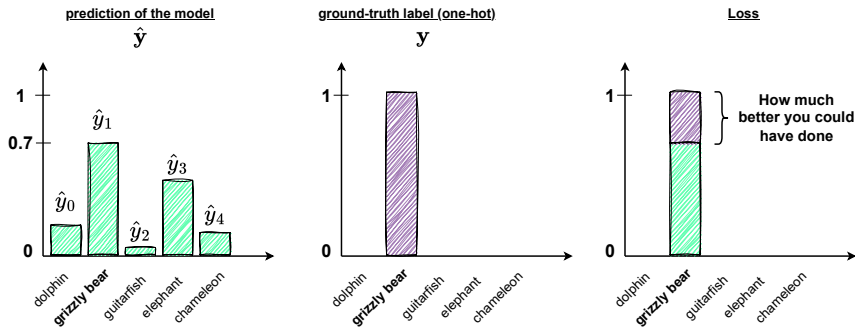
$$\mathbf{z} = z_{\Theta}(\mathbf{x})$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}),$$

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{i=1}^K e^{z_i}}$$

where $\mathbf{x}$ is the input image.

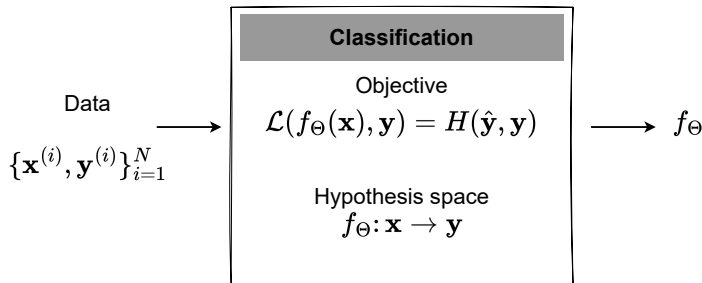
Classification Problem: Cross-Entropy Loss



- The prediction placed around 70% (or 0.7) probability on the true class "grizzly bear", so we are 30% (or 0.3) off from an ideal prediction.
- Therefore, our loss is $-\log 0.7$ (Recall the formula: $-\sum_{k=1}^K y_k \log \hat{y}_k$).

Classification Problem

- We did not discuss a couple of things: hypothesis space and optimizer. What about them?
 - ▶ Hypothesis space will be discussed in the Neural Network lectures
 - ▶ Optimizer remains the same as linear regression problem



Extra Readings

- [Dive into Deep Learning](https://d2l.ai/index.html) (<https://d2l.ai/index.html>). Chapter 4 (4.1 and 4.4)

Credits and Acknowledgements

- Some slides have been adapted from Torralba, Isola and Freeman